

Running the Course

Automation, grading, and tools

Routine Automation

- The repetitive course tasks are scripts behind one runner
- `just` lists them; you run them by name
- The agent wrote most of these scripts in the first place

```
$ just --list
gen-pdfs      lecture PDFs
gen-notes     podium notes
ss            schedule stream
get-transcripts
gen-profiles  roster from Canvas
autograde     run tests locally
```

Scripts Behind just

- Python scripts, each with one job
- Path-independent, so they run from anywhere in the repo
- Add a script, add a recipe, and it joins the menu

Script	Job
<code>generate_pdfs</code>	slides to PDF
<code>get_transcripts</code>	pull YouTube
<code>schedule_stream</code>	book a stream
<code>gen_profiles</code>	roster CSV
<code>autograder</code>	grade repos

The Autograder

- Runs the test suite on every student push
- Publishes results as a GitHub release
- Feedback is automatic, so homework can be for learning

```
on: [push]

jobs:
  grade:
    run: python autograder.py run -l
    # results.md -> release
```

Homework as Learning

- If the grade is not the point, the work can be
- Students get instant feedback and keep going
- You have to write the code to learn to read it

```
push -> tests run -> feedback  
      -> fix -> push again
```

```
the loop is the lesson
```

Tools and Visualizers

- The agent builds the small tools the course needs
- Canvas helper for bulk grade entry
- MTMC, BDP, and bytecode visualizers for the lectures

<code>canvas-helper</code>	bulk grades
<code>mtmc-web</code>	CPU emulator
<code>bdp-1</code>	8-bit model
<code>bytecode-viz</code>	step through

main vs student Branches

- `main` holds working implementations -- the tests pass
- `student` ships stubs marked `// TODO - implement`
- Students fill in the code and make the tests pass

main	student
----	-----
full	// TODO - implement
impl	describe the result,
tests	not the steps
pass	tests fail (on purpose)

Keeping Branches in Sync

- Merge `main` into `student`, then strip implementations
- Delete do-not-ship content: keys, notes, solutions
- The agent does the mechanical strip; you verify nothing leaked

```
1. merge main -> student
2. stub implementations
3. drop keys + notes
4. grep for leaked solutions
5. tests compile, still fail
```


AI as TA, Not Answer Key

- Students get an AGENTS.md that frames the agent as a tutor
- It explains and unblocks; it does not hand over solutions
- The temptation is real -- the structure pushes against it

```
# AGENTS.md
```

- * You are a teaching assistant
- * Explain concepts, do not write the student's solution
- * Code examples: 2-5 lines max

DEMO: just in Action

- Show the full recipe menu with `just --list`
- Run PDF generation end to end
- Point out that one command wraps a whole script

Summary

- Repetitive course tasks are scripts behind `just` recipes
- The agent wrote most of those scripts to begin with
- The autograder runs tests on every push and publishes results
- Automatic feedback lets homework be for learning, not just grading
- The agent builds the course's tools and visualizers
- `main` carries implementations; `student` ships stubs
- Syncing merges main, strips solutions, and verifies no leaks
- AGENTS.md frames the agent as a tutor, not an answer key

Running the Course

Automation, grading, and tools
